



eSigned on **LegalZoom** 62.215.38.249
Fitzgerald
J. Heslop 06:14:40 GMT
Sep 19, 2026

```
import A5O
set_option autoImplicit false
```

```
/-! # A5O — Kill-switch survival, over v4's own predicates (2026-09-19).
Imports A5O.lean v4 (SHA-256 0e5ebe3f...5b452) unmodified. Restates the
T6/T7 content of the standalone KillSwitch.lean (d55dfa11...) as theorems
about v4's `O`, `Appointed`, `R`, and `Revoke`, with revocation dominance
in v4's own P2 shape. Zero axioms.
```

```
Model of compromise: `CompP P` — the adversary controls principal P;
`CompA A` — the adversary controls agent A. The channel rule: an intended
revocation is either on the ledger or was suppressed, and suppression is
compromise of the issuer; a revocation on the ledger is never from a
compromised issuer. An agent-keyed switch obeys the same rule keyed on
the agent. Nothing about the agent's compromise appears in P2 — that is
the whole point. -/
```

```
namespace A5O.KillSwitchA5O
```

```
variable {Principal Agent Action Time : Type} [LT Time]
variable (C : Principal → Agent → Action → Time → Prop)
variable (I : Principal → Prop)
variable (M : Agent → Action → Prop)
variable (S : Agent → Action → Prop)
variable (R : Principal → Agent → Action → Time → Prop)
variable (Appointed : Principal → Agent → Action → Time → Prop)
variable (Revoke : Principal → Agent → Action → Time → Prop)
variable (Intends : Principal → Agent → Action → Time → Prop)
variable (CompP : Principal → Prop)
variable (CompA : Agent → Prop)
```

```
/-- v4's P2 shape, exactly as `revoked_defeats_O` consumes it. -/
```

```
def P2 : Prop :=
  ∀ (P : Principal) (A : Agent) (X : Action) (t t' : Time),
    t < t' → Appointed P A X t → Revoke P A X t' → ¬ R P A X t'
```

```
/-- Channel, stated positively (constructive):
```

- (a) an intended revocation reaches the ledger unless its issuer is compromised;
- (b) a revocation on the ledger is not from a compromised issuer. -/

```
def Channel : Prop :=
  (∀ (P : Principal) (A : Agent) (X : Action) (t : Time),
    Intends P A X t → Revoke P A X t ∨ CompP P) ∧
  (∀ (P : Principal) (A : Agent) (X : Action) (t : Time),
    Revoke P A X t → ¬ CompP P)
```

```
-- =====
-- KS1. Appointer-keyed revocation survives ANY compromise of the agent.
-- =====
```

```
-- After a revocation on the ledger, O fails — and no hypothesis about the
agent's compromise is needed: `CompA` does not occur. Revocation dominance
read as a security property. -/
```

```
theorem revocation_survives_agent_compromise
  (hP2 : P2 R Appointed Revoke)
  (P : Principal) (A : Agent) (X : Action) (t t' : Time)
  (hlt : t < t') (hApp : Appointed P A X t) (hRev : Revoke P A X t') :
  ¬ O C I M S R Appointed P A X t' :=
  revoked_defeats_O C I M S R Appointed Revoke hP2 P A X t t' hlt hApp hRev
```

```
-- Same, with the agent's full compromise stated and then unused, to make
the independence visible in the statement. -/
```

```
theorem revocation_survives_full_agent_compromise
  (hP2 : P2 R Appointed Revoke)
  (P : Principal) (A : Agent) (X : Action) (t t' : Time)
  (_hFull : CompA A)
  (hlt : t < t') (hApp : Appointed P A X t) (hRev : Revoke P A X t') :
  ¬ O C I M S R Appointed P A X t' :=
  revoked_defeats_O C I M S R Appointed Revoke hP2 P A X t t' hlt hApp hRev
```

```
-- =====
-- KS2. The boundary: the switch fails exactly when the appointer is
--   compromised.
-- =====
```

```
-- Sufficiency: intent from an uncompromised appointer defeats O. -/
```

```
theorem honest_appointer_kills
  (hP2 : P2 R Appointed Revoke) (hCh : Channel Revoke Intends CompP)
  (P : Principal) (A : Agent) (X : Action) (t t' : Time)
  (hlt : t < t') (hApp : Appointed P A X t)
  (hInt : Intends P A X t') (hHonest : ¬ CompP P) :
  ¬ O C I M S R Appointed P A X t' :=
  fun hO =>
  match hCh.1 P A X t' hInt with
  | Or.inl hRev =>
    revoked_defeats_O C I M S R Appointed Revoke hP2 P A X t t' hlt hApp hRev hO
  | Or.inr hComp => hHonest hComp
```

```
-- Necessity: if the appointer intended revocation and O nonetheless holds
```

afterward, the appointer was compromised. Inside the model there is no other way for an intended revocation to fail. -/

theorem defeat_implies_appointer_compromised

(hP2 : P2 R Appointed Revoke) (hCh : Channel Revoke Intends CompP)

(P : Principal) (A : Agent) (X : Action) (t' : Time)

(hlt : t < t') (hApp : Appointed P A X t)

(hInt : Intends P A X t') (hO : O C I M S R Appointed P A X t') :

CompP P :=

match hCh.1 P A X t' hInt with

| Or.inl hRev =>

absurd hO (revoked_defeats_O C I M S R Appointed Revoke hP2 P A X t' hlt hApp hRev)

| Or.inr hComp => hComp

/-- The biconditional, over an appointed agent with intent to revoke:

O survives the intended revocation $\leftrightarrow$ the appointer is compromised. -/

theorem survival_iff_appointer_compromised

(hP2 : P2 R Appointed Revoke) (hCh : Channel Revoke Intends CompP)

(P : Principal) (A : Agent) (X : Action) (t' : Time)

(hlt : t < t') (hApp : Appointed P A X t) (hInt : Intends P A X t') :

(O C I M S R Appointed P A X t' $\rightarrow$ CompP P) $\wedge$

($\neg$ CompP P $\rightarrow \neg$ O C I M S R Appointed P A X t') :=

(defeat_implies_appointer_compromised C I M S R Appointed Revoke Intends CompP

hP2 hCh P A X t' hlt hApp hInt,

honest_appointer_kills C I M S R Appointed Revoke Intends CompP

hP2 hCh P A X t' hlt hApp hInt)

-- =====
 -- KS3. Contrast: an agent-keyed switch draws from the agent's well.
 -- =====

section SelfKeyed

set_option linter.unusedSectionVars false

variable (SelfKill : Agent $\rightarrow$ Action $\rightarrow$ Time $\rightarrow$ Prop)

variable (SelfIntends : Agent $\rightarrow$ Action $\rightarrow$ Time $\rightarrow$ Prop)

/-- The same channel rule, keyed on the agent. -/

def SelfChannel : Prop :=

($\forall$ (A : Agent) (X : Action) (t : Time),

SelfIntends A X t $\rightarrow$ SelfKill A X t $\vee$ CompA A) $\wedge$

($\forall$ (A : Agent) (X : Action) (t : Time), SelfKill A X t $\rightarrow \neg$ CompA A)

/-- Full compromise of the agent silences every agent-keyed switch:

no SelfKill can appear on the ledger for a compromised agent. -/

theorem self_keyed_silenced_by_agent_compromise

(hSC : SelfChannel CompA SelfKill SelfIntends)

```

(A : Agent) (X : Action) (t : Time) (hFull : CompA A) :
  ¬ SelfKill A X t :=
fun hK => hSC.2 A X t hK hFull

/-- And therefore an agent-keyed switch gives nothing against a compromised
agent even with intent: the only remaining branch is compromise itself. -/
theorem self_keyed_intent_useless_under_compromise
  (hSC : SelfChannel CompA SelfKill SelfIntends)
  (A : Agent) (X : Action) (t : Time) (hFull : CompA A)
  (_hInt : SelfIntends A X t) :
  ¬ SelfKill A X t :=
  self_keyed_silenced_by_agent_compromise CompA SelfKill SelfIntends hSC A X t hFull
end SelfKeyed

end A5O.KillSwitchA5O

-- =====
-- Witness: the hypotheses are jointly satisfiable and KS2 is not vacuous.
-- =====
namespace A5O.KillSwitchA5OWitness
open A5O.KillSwitchA5O

def kC : Unit → Unit → Unit → Nat → Prop := fun _ _ _ => True
def kI : Unit → Prop := fun _ => True
def kM : Unit → Unit → Prop := fun _ _ => True
def kS : Unit → Unit → Prop := fun _ _ => True
def kApp : Unit → Unit → Unit → Nat → Prop := fun _ _ _ => True
/- The principal intends revocation from time 1. -/
def kInt : Unit → Unit → Unit → Nat → Prop := fun _ _ _ t => 1 ≤ t
/- Honest principal: the revocation is on the ledger from time 1. -/
def kRev : Unit → Unit → Unit → Nat → Prop := fun _ _ _ t => 1 ≤ t
/- Revocation dominance: R stands only before time 1. -/
def kR : Unit → Unit → Unit → Nat → Prop := fun _ _ _ t => t = 0
def kCompP : Unit → Prop := fun _ => False
/- The agent is fully compromised throughout. -/
def kCompA : Unit → Prop := fun _ => True

theorem k_P2 : P2 kR kApp kRev := by
  intro _ _ _ t' _ _ hRev hR
  -- hRev : 1 ≤ t', hR : t' = 0
  have h : (1:Nat) ≤ 0 := hR ▶ hRev
  exact Nat.not_succ_le_zero 0 h

theorem k_Channel : Channel kRev kInt kCompP :=

```

```
⟨fun _____ h => Or.inl h, fun _____ h => h⟩
```

```
-- O holds at 0 (the receipt), and the compromised agent cannot keep it past 1. -/
```

```
theorem k_O_at_0 : O kC kl kM kS kR kApp () () () 0 :=
```

```
⟨trivial, trivial, trivial, trivial, rfl, trivial⟩
```

```
theorem k_killed_at_1 : ¬ O kC kl kM kS kR kApp () () () 1 :=
```

```
revocation_survives_full_agent_compromise kC kl kM kS kR kApp kRev kCompA k_P2  
() () () 0 1 trivial (Nat.lt_succ_self 0) trivial (Nat.le_refl 1)
```

```
end A5O.KillSwitchA5OWitness
```

```
#print axioms A5O.KillSwitchA5O.revocation_survives_agent_compromise
```

```
#print axioms A5O.KillSwitchA5O.revocation_survives_full_agent_compromise
```

```
#print axioms A5O.KillSwitchA5O.honest_appointer_kills
```

```
#print axioms A5O.KillSwitchA5O.defeat_implies_appointer_compromised
```

```
#print axioms A5O.KillSwitchA5O.survival_iff_appointer_compromised
```

```
#print axioms A5O.KillSwitchA5O.self_keyed_silenced_by_agent_compromise
```

```
#print axioms A5O.KillSwitchA5O.self_keyed_intent_useless_under_compromise
```

```
#print axioms A5O.KillSwitchA5OWitness.k_P2
```

```
#print axioms A5O.KillSwitchA5OWitness.k_killed_at_1
```